Caesar Cipher

Web Application — Technical Report

Version 0.0.1 | 2026-10-09 | Author: Victor Barrientos

1. Overview

A static web application, built with Astro 7.3.6, that encrypts and decrypts messages with the Caesar cipher. Users pick an action (Encrypt/Decrypt), a vocabulary (English/Spanish), type a message, set a shift between 3 and 7, and copy the result. Styling is entirely Tailwind CSS v4, and the layout is responsive.

Architecture

- **src/classes/Caesar.js** — pure cipher logic; a static class exported as an ES module.
- **src/components/CaesarCipher.astro** — interactive card; its bundled `script` imports `Caesar` and wires buttons, form and clipboard.
- **src/components/Main.astro** — header/navigation and hero section.
- **src/layouts/Layout.astro** — HTML shell, global CSS import, floating footer.
- **src/pages/** — `index.astro` (home) and `manual.astro` (user manual); **tests/Caesar.test.js** — Vitest suite.

2. Implemented Logic

The core is a single static class in `Caesar.js` whose behaviour is fully driven by private static fields:

Field	Default	Purpose
#englishABC	a..z (26)	English alphabet
#spanishABC	a..z + ñ (27)	Spanish alphabet, ñ inserted after n
#encryptMode	true	Direction: encrypt (1) vs decrypt (-1)
#useEnglishAlphabet	true	Selected vocabulary
#minValue / #maxValue	3 / 7	Valid shift range

The cipher algorithm

`cipher(text, shift)` validates the inputs, normalises a signed shift, and maps every character through the selected alphabet:

```
if (shift > maxValue || shift < minValue || text.length === 0) return "";

const abc = Caesar.getABC();                    // english or spanish alphabet
const k   = (((shift * direction) % n) + n) % n;   // positive, sign folded

return [...text].map(ch => {
    const i = abc.indexOf(ch.toLowerCase());
    if (i === -1) return ch;                    // non-letter: unchanged
    const m = abc[(i + k) % n];
    return ch === ch.toLowerCase() ? m : m.toUpperCase();
}).join("");
```

Key properties: **validation first** (out-of-range shift or empty text returns ""), **positive modulo** — the direction is folded into `k`, giving correct wrap-around at both ends of the alphabet, **case preservation**, and **non-letters pass through**. The Spanish alphabet is derived by inserting "ñ" after "n" with `toSpliced` (27 letters).

3. Design Decisions

Logic-layer decisions

- **Static class, no instantiation.** The cipher is stateless from a caller's perspective; all state lives in static private fields (#), so the module exposes a single shared `Caesar` object. This lets the same file be reused verbatim by the browser bundle and by the Vitest suite — one source of truth.
- **Private fields for encapsulation.** The alphabets, mode flags and the shift range are truly private; consumers can only interact through the documented setter methods and `cipher`.
- **Restricted shift range (3–7).** A shift of 0 or 1 would make the cipher trivial, so input is confined to a meaningful range and validated both in the UI (min/max attributes) and in the class itself.
- **Early validation.** `text.length === 0` and out-of-range shift return an empty string before any traversal, keeping the algorithm branch-light and predictable.
- **Direction folded into the shift.** A single positive k serves both modes, eliminating a whole class of modulo-edge bugs discovered during testing.
- **ESM everywhere.** `"type": "module"` makes the class importable from Astro scripts and tests without transpilation.

UI / UX decisions

- **Astro plus a bundled script, no framework island.** The interactivity is pure vanilla JavaScript shipped by Astro's script bundler — no hydration, no runtime framework, tiny payload.
- **Tailwind CSS v4 through `@tailwindcss/vite`.** A single `@import "tailwindcss";` in `global.css`; all styling is utility classes and no scoped `<style>` blocks remain, keeping the design consistent across components.
- **Minimal dark UI.** A slate background with one subtle indigo glow; the cipher card is a static "floating" glass element (`backdrop-blur`, translucent background, soft border and shadow) — deliberately not animated.
- **Segmented toggles with an explicit active state.** The Action and Alphabet buttons are toggles; the selected one is highlighted (indigo) via a `setActiveButton` helper, and a summary panel mirrors the current state for user confidence.
- **Visible, labelled inputs.** "Text to cipher", "Shift (3 - 7)" and "Result" all use real `<label>` elements, and the two inputs are marked `required` to prevent empty submits.
- **Clipboard on demand.** A "Copy result" button is revealed on the first submit that yields a non-empty result (`copyButtonShown` flag) and gives explicit success/error feedback before reverting.
- **Data attributes for dynamic labels.** The result label switches between "Ciphertext" and "Plaintext" using `data-label-cipher/data-label-decipher`.
- **Responsive layout.** Mobile-first grids (`sm:grid-cols-2/3`), a fluid hero, and a footer pinned to the bottom with `mt-auto`.
- **Documentation page.** `/manual` is a real user manual matching the shipped controls, and the header exposes Technical Report / User Manual / Source Code links.

4. Test Results

The suite is executed with **Vitest 5.0.3** (`pnpm test`) against the pure `Caesar` class, importing the production module unchanged. A `beforeEach` hook resets the static state to Encrypt + English alphabet so tests are isolated.

Test suite	Tests	Result
Caesar mode setters	5	5 / 5 passed
Caesar.cipher encryption	5	5 / 5 passed
Caesar.cipher decryption	3	3 / 3 passed
Caesar.cipher Spanish vocabulary	3	3 / 3 passed
Caesar.cipher shift validation	6	6 / 6 passed
Total	**22**	**22 / 22 passed**

Last full run: **1 test file**, **22 tests passed** — 0 failed, 0 skipped.

Test coverage highlights

- **Mode setters (5).** Defaults, decrypt flips behaviour, encrypt after decrypt, and switching between alphabets in both directions.
- **Encryption (5).** Simple shift, end-of-alphabet wrap-around (`xyz→abc`), case preservation, and non-letters (spaces, digits, punctuation) pass through unchanged.
- **Decryption (3).** Backward shift, start-of-alphabet wrap-around (`abc→xyz`), and a full English round-trip (`"The quick brown fox"` with shift 5).
- **Spanish vocabulary (3).** ñ treated as a regular letter (`ñ→q`), wrap before `a`, and a Spanish round-trip that preserves case and numbers.
- **Shift validation (6).** Values 0, 2, 8 are rejected with ""; boundaries 3 and 7 are accepted; empty text returns "".

Testing value added

The suite caught a real defect early: decrypting letters near the start of the alphabet produced negative indices and silently dropped characters (e.g. `"The quick brown fox"` decrypted to `"The quick bron fo"`). Normalising the shift to a positive `k` fixed it, and the wrap-around decrypt tests now guard against regression. The same tests serve as living documentation of the class contract.

How to run

```
pnpm install     # install dependencies
pnpm test        # run Vitest suites
pnpm build      # build the Astro site
pnpm dev        # start the dev server
```

Generated from the project codebase at https://github.com/barrientosvctor/caesar-cipher.